Driver Analysis Document

Embedded Software Project: STM32 Peripheral Drivers

Target MCU: STM32F446RE (Nucleo-F446RE)

Implemented Peripherals: GPIO, USART, SPI

Target Application: ADXL345 Accelerometer via SPI

Author: Sreeraj Krishna K

Date: June 2026

1. General Architecture & Design Philosophy

This project utilizes a bare-metal, register-level approach to configuring the STM32F446RE microcontroller. Rather than relying on the STMicroelectronics Hardware Abstraction Layer (HAL), these drivers interact directly with the memory-mapped peripheral registers defined in the CMSIS core headers (stm32f446xx.h). This approach guarantees minimal overhead, deterministic execution, and provides a deep understanding of the internal hardware mechanics of the ARM Cortex-M4 architecture.

The overall design choice across all implemented drivers (GPIO, SPI, USART) is a blocking, polling-based methodology. While interrupt-driven (NVIC) or DMA-based approaches offer superior CPU efficiency, polling is deliberately chosen here to abstract complexity, ensuring linear and highly predictable code execution for basic sensor data acquisition (like the ADXL345) and debugging outputs.

2. GPIO (General Purpose Input/Output) Driver

2.1 Hardware Abstraction & Initialization

The GPIO driver abstracts the control of physical MCU pins. Initialization is performed by manipulating the target port's configuration registers:

- Clock Enable: The AHB1 peripheral clock enable register (RCC->AHB1ENR) is modified to route clock signals to the specific GPIO port (e.g., Port A, Port C). Without this, the peripheral is unpowered and unresponsive to register writes.
- Mode Configuration: The MODER register is configured to set pins as Input (00), Output (01), Alternate Function (10), or Analog (11). In this project, pins are set as General Purpose Outputs (e.g., PC0, PC1, PC2 for LED indicators) or Alternate Functions (for SPI and USART routing).
- Speed and Pull-up/Pull-down: Though not heavily utilized in the basic output examples, the OSPEEDR and PUPDR registers allow fine-tuning of the electrical characteristics of the pins, which is critical for high-speed buses like SPI.

2.2 Data I/O and Usage

Writing data to a pin is accomplished by modifying the Output Data Register (ODR) or the Bit Set/Reset Register (BSRR). The BSRR provides an atomic way to change pin states without the read-modify-write risks associated with the ODR. In the project's example application, GPIO is

heavily utilized to drive LED feedback based on sensor tilt thresholds, demonstrating rapid state toggling and basic software delay loops.

3. SPI (Serial Peripheral Interface) Driver

3.1 Initialization and Configuration

The SPI driver is configured to act as the Master device to communicate with the ADXL345 accelerometer. Initialization involves:

- GPIO Alternate Function: The SCK (Clock), MISO (Master In Slave Out), and MOSI (Master Out Slave In) pins are configured to Alternate Function mode, routing their control from standard GPIO logic directly to the internal SPI hardware block.
- Peripheral Clocking: The APB2 peripheral clock (RCC->APB2ENR) is enabled for SPI1.
- Control Register 1 (CR1): This register is the core of the SPI configuration. The MCU is set to Master Mode, the Baud Rate (BR) prescaler is set to ensure the clock speed does not exceed the slave device's limits, and the Data Frame Format (DFF) is kept at 8-bit. Furthermore, Clock Polarity (CPOL) and Clock Phase (CPHA) are configured to match the ADXL345's requirements (typically CPOL=1, CPHA=1 for Mode 3).

3.2 Data Transfer Mechanism

The driver uses a polling mechanism for both transmission and reception:

- Transmission (spi1_transmit): The software loads a byte into the Data Register (SPI->DR). It then polls the Status Register (SPI->SR), specifically waiting in a while loop until the Transmit Buffer Empty (TXE) flag is set before loading the subsequent byte.
- Reception (spi1_receive): To receive data, the Master must generate clock cycles. The driver sends a dummy byte to trigger the clock, then polls the Receive Buffer Not Empty (RXNE) flag before reading the incoming byte from SPI->DR.

Design Choice - Software Slave Management: The Chip Select (CS) pin is explicitly not managed by the SPI hardware's NSS feature. Instead, it is configured as a standard GPIO output. This allows the software to pull CS LOW (cs_enable()) before a multi-byte transaction and pull it HIGH (cs_disable()) afterward, providing precise control necessary for the ADXL345's burst read sequence.

4. USART (Universal Synchronous/Asynchronous Receiver Transmitter) Driver

4.1 Initialization and Configuration

The USART driver is implemented to provide a debugging console via the Nucleo board's integrated ST-LINK virtual COM port (USART2 connected to PA2/TX and PA3/RX).

- Clock and Pin Setup: APB1 clock is enabled for USART2. PA2 and PA3 are set to Alternate Function mode.
- Baud Rate Calculation: The driver computes the necessary value for the Baud Rate Register (USART_BRR) based on the peripheral clock frequency and the desired baud rate (e.g., 115200). This involves calculating the USARTDIV mantissa and fraction components.
- Control Registers: The CR1 register is configured to enable the USART peripheral (UE), and enable both the Transmitter (TE) and Receiver (RE). The word length is kept at the default 8 data bits, 1 start bit, 1 stop bit, with no parity.

4.2 Data Transfer Mechanism

Similar to SPI, USART transmission is polling-based. The uart_write() function waits for the TXE flag in the Status Register before loading a character into the Data Register.

Integration with Standard C Library: A critical architectural feature of this project is the redirection of the standard printf function. By overriding the weak _write system call within syscalls.c to point to our custom uart_write() function, the application code can output floating-point numbers, strings, and formatted accelerometer data directly to a PC serial terminal seamlessly.

5. I2C (Inter-Integrated Circuit) Driver - Architectural Overview

Note: While the primary application in this codebase utilizes SPI for sensor communication, the I2C driver architecture follows a similar bare-metal paradigm.

An I2C driver requires configuring pins as Open-Drain (unlike the Push-Pull used for SPI) with internal or external pull-up resistors. Initialization involves setting the CR1, CR2, CCR (Clock Control Register), and TRISE registers to achieve standard (100kHz) or fast (400kHz) mode. Data transfer in I2C is state-heavy, requiring the software to manually trigger Start conditions, transmit 7-bit slave

addresses with R/W bits, poll for ACK/NACK responses, read/write the Data Register, and finally generate Stop conditions. This half-duplex communication requires more meticulous flag checking (TXE, RXNE, BTF, ADDR) compared to the full-duplex SPI.